



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 2
по курсу «Разработка параллельных и распределенных
программ»

«Параллельная реализация решения системы линейных
алгебраических уравнений с помощью MPI»

Студент группы ИУ9-51Б Григорьева С. А.

Преподаватель Царев А. С.

Москва 2025

1 Задача

1. Написать программу, которая реализует итерационный алгоритм решения системы линейных алгебраических уравнений вида $Ax=b$ в соответствии с выбранным вариантом. Здесь A – матрица размером $N \times N$, x и b – векторы длины N . Тип элементов – `double`.

2. Программу распараллелить с помощью MPI с разрезанием матрицы A по строкам на близкие по размеру, возможно не одинаковые, части. Соседние строки матрицы должны располагаться в одном или в соседних MPI-процессах. Реализовать два варианта программы:

1: векторы x и b дублируются в каждом MPI-процессе,

2: векторы x и b разрезаются между MPI-процессами аналогично матрице A . Уделить внимание тому, чтобы при запуске программы на различном числе MPI-процессов решалась одна и та же задача (исходные данные заполнялись одинаковым образом).

3. Замерить время работы двух вариантов программы при использовании различного числа процессорных ядер: 1, 2, 4, 8, 16. Построить график зависимости времени работы программы, ускорения и эффективности распараллеливания от числа используемых ядер. Исходные данные, параметры N и ε подобрать таким образом, чтобы решение задачи на одном ядре занимало не менее 30 секунд. Также параметр N разрешено подобрать таким образом, чтобы он нацело делился на 1, 2, 4, 8 и 16.

4. На основании полученных результатов сделать вывод о целесообразности использования одного или второго варианта программы.

Метод минимальных невязок

В методе минимальных невязок преобразование решения на каждом шаге задается следующими формулами:

$$\begin{aligned}y^n &= Ax^n - b, \\ \tau^n &= \frac{(y^n, Ay^n)}{(Ay^n, Ay^n)}, \\ x^{n+1} &= x^n - \tau^n y^n,\end{aligned}$$

где $(u, v) = \sum_{i=0}^{N-1} u_i v_i$. В отличие от метода простой итерации, метод минимальных невязок содержит два умножения матрицы на вектор на каждой итерации, однако сходится до нужной точности за меньшее число итераций. Критерий завершения счета можно взять такой же, как в методе простой итерации.

2 Практическая реализация

Исходный код программы представлен в листингах 1– 2– 3– 4.

Листинг 1 — Вариант 1 - часть 1

```

1 #include <mpi.h>
2 #include <cmath>
3 #include <vector>
4 #include <iostream>
5 #include <random>
6
7 constexpr int N = 60000;
8 constexpr double EPS = 1e-5;
9 constexpr int MAX_ITER = 100;
10
11 struct RowRange {
12     int start;
13     int end;
14 };
15
16 RowRange get_range(int rank, int size, int N_total) {
17     int base = N_total / size;
18     int rem = N_total % size;
19     RowRange r;
20     if (rank < rem) {
21         r.start = rank * (base + 1);
22         r.end = r.start + (base + 1);
23     } else {
24         r.start = rem * (base + 1) + (rank - rem) * base;
25         r.end = r.start + base;
26     }
27     return r;
28 }
29
30 int main(int argc, char** argv) {
31     MPI_Init(&argc, &argv);
32     int rank, size;
33     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
34     MPI_Comm_size(MPI_COMM_WORLD, &size);
35     RowRange range = get_range(rank, size, N);
36     int local_rows = range.end - range.start;
37     std::vector<double> x(N, 0.0);
38     std::vector<double> b(N, 0.0);
39     std::vector<double> y(N, 0.0);
40     std::vector<double> tmpY(local_rows);
41     std::vector<double> Ay_local(local_rows);
42     std::vector<double> Ay(N);
43     if (rank == 0) {
44         std::mt19937 gen(42);
45         std::uniform_real_distribution<double> dist(0.0, 1.0);
46         for (int i = 0; i < N; ++i) b[i] = dist(gen);
47     }
48     MPI_Bcast(b.data(), N, MPI_DOUBLE, 0, MPI_COMM_WORLD);
49     double local_b2 = 0.0;
50     for (int i = range.start; i < range.end; ++i) local_b2 += b[i] * b[i];
51     double b2 = 0.0;
52     MPI_Allreduce(&local_b2, &b2, 1, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
53     double b_norm = std::sqrt(b2);
54     if (b_norm == 0.0) {
55         MPI_Finalize();
56         return 0;
57     }

```

Листинг 2 — Вариант 1 - часть 2

```

1  std::vector<int> counts(size), displs(size);
2  for (int p = 0; p < size; ++p) {
3      RowRange r = get_range(p, size, N);
4      counts[p] = r.end - r.start;
5      displs[p] = r.start;
6  }
7  int iter = 0;
8  double rel_res = 0.0;
9  double t_start = MPI_Wtime();
10 while (iter < MAX_ITER) {
11     for (int li = 0; li < local_rows; ++li) {
12         int i = range.start + li;
13         double sum = 2.0 * x[i];
14         for (int j = 0; j < N; ++j) if (i != j) sum += x[j];
15         tmpY[li] = sum - b[i];
16     }
17     MPI_Allgatherv(tmpY.data(), local_rows, MPI_DOUBLE,
18                   y.data(), counts.data(), displs.data(),
19 MPI_DOUBLE, MPI_COMM_WORLD);
20     double local_r2 = 0.0;
21     for (int li = 0; li < local_rows; ++li) local_r2 += tmpY[li] *
22 tmpY[li];
23     double r2 = 0.0;
24     MPI_Allreduce(&local_r2, &r2, 1, MPI_DOUBLE, MPI_SUM,
25 MPI_COMM_WORLD);
26     rel_res = std::sqrt(r2) / b_norm;
27     if (rel_res < EPS) break;
28     for (int li = 0; li < local_rows; ++li) {
29         int i = range.start + li;
30         double sum = 2.0 * y[i];
31         for (int j = 0; j < N; ++j) if (i != j) sum += y[j];
32         Ay_local[li] = sum;
33     }
34     MPI_Allgatherv(Ay_local.data(), local_rows, MPI_DOUBLE,
35                   Ay.data(), counts.data(), displs.data(),
36 MPI_DOUBLE, MPI_COMM_WORLD);
37     double local_num = 0.0, local_den = 0.0;
38     for (int li = 0; li < local_rows; ++li) {
39         int i = range.start + li;
40         local_num += y[i] * Ay[i];
41         local_den += Ay[i] * Ay[i];
42     }
43     double num = 0.0, den = 0.0;
44     MPI_Allreduce(&local_num, &num, 1, MPI_DOUBLE, MPI_SUM,
45 MPI_COMM_WORLD);
46     MPI_Allreduce(&local_den, &den, 1, MPI_DOUBLE, MPI_SUM,
47 MPI_COMM_WORLD);
48     if (den == 0.0) break;
49     double tau = num / den;
50     for (int i = 0; i < N; ++i) x[i] -= tau * y[i];
51     ++iter;
52 }
53 double t_end = MPI_Wtime();
54 if (rank == 0) {
55     std::cout << "Time: " << (t_end - t_start);
56 }
57 MPI_Finalize();
58 return 0;
59 }

```

Листинг 3 — Вариант 2 - часть 1

```

1 #include <mpi.h>
2 #include <cmath>
3 #include <vector>
4 #include <iostream>
5 #include <random>
6 constexpr int N = 60000;
7 constexpr double EPS = 1e-5;
8 constexpr int MAX_ITER = 100;
9 struct RowRange {
10     int start;
11     int end;
12 };
13 RowRange get_range(int rank, int size, int N_total) {
14     int base = N_total / size;
15     int rem = N_total % size;
16     RowRange r;
17     if (rank < rem) {
18         r.start = rank * (base + 1);
19         r.end = r.start + (base + 1);
20     } else {
21         r.start = rem * (base + 1) + (rank - rem) * base;
22         r.end = r.start + base;
23     }
24     return r;
25 }
26 int main(int argc, char** argv) {
27     MPI_Init(&argc, &argv);
28     int rank, size;
29     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
30     MPI_Comm_size(MPI_COMM_WORLD, &size);
31     RowRange range = get_range(rank, size, N);
32     int local_rows = range.end - range.start;
33     std::vector<double> x_local(local_rows, 0.0);
34     std::vector<double> b_local(local_rows);
35     std::vector<double> y_local(local_rows);
36     std::vector<double> Ay_local(local_rows);
37     std::vector<double> x_full(N);
38     std::vector<double> y_full(N);
39     if (rank == 0) {
40         std::mt19937 gen(42);
41         std::uniform_real_distribution<double> dist(0.0, 1.0);
42         std::vector<double> b_full(N);
43         for (int i = 0; i < N; ++i) b_full[i] = dist(gen);
44         std::vector<int> counts(size), displs(size);
45         for (int r = 0; r < size; ++r) {
46             RowRange rr = get_range(r, size, N);
47             counts[r] = rr.end - rr.start;
48             displs[r] = rr.start;
49         }
50         MPI_Scatterv(b_full.data(), counts.data(), displs.data(),
MPI_DOUBLE, b_local.data(), local_rows, MPI_DOUBLE, 0,
MPI_COMM_WORLD);
51     } else {
52         std::vector<int> counts(size), displs(size);
53         for (int r = 0; r < size; ++r) {
54             RowRange rr = get_range(r, size, N);
55             counts[r] = rr.end - rr.start;
56             displs[r] = rr.start;
57         }

```

Листинг 4 — Вариант 2 - часть 2

```

1      MPI_Scatterv(nullptr, counts.data(), displs.data(), MPI_DOUBLE,
2      b_local.data(), local_rows, MPI_DOUBLE, 0, MPI_COMM_WORLD);
3      }
4      double local_b2 = 0;
5      for (double bi : b_local) local_b2 += bi * bi;
6      double b2 = 0;
7      MPI_Allreduce(&local_b2, &b2, 1, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
8      double b_norm = sqrt(b2);
9      int iter = 0;
10     double rel_res = 0;
11     double t_start = MPI_Wtime();
12     while (iter < MAX_ITER) {
13         MPI_Allgather(x_local.data(), local_rows, MPI_DOUBLE,
14                       x_full.data(), local_rows, MPI_DOUBLE,
15                       MPI_COMM_WORLD);
16         for (int li = 0; li < local_rows; ++li) {
17             int i = range.start + li;
18             double s = 2.0 * x_full[i];
19             for (int j = 0; j < N; ++j) if (i != j) s += x_full[j];
20             y_local[li] = s - b_local[li];
21         }
22         double local_r2 = 0;
23         for (double yi : y_local) local_r2 += yi * yi;
24         double r2 = 0;
25         MPI_Allreduce(&local_r2, &r2, 1, MPI_DOUBLE, MPI_SUM,
26                       MPI_COMM_WORLD);
27         rel_res = sqrt(r2) / b_norm;
28         if (rel_res < EPS) break;
29         MPI_Allgather(y_local.data(), local_rows, MPI_DOUBLE,
30                       y_full.data(), local_rows, MPI_DOUBLE,
31                       MPI_COMM_WORLD);
32         for (int li = 0; li < local_rows; ++li) {
33             int i = range.start + li;
34             double s = 2.0 * y_full[i];
35             for (int j = 0; j < N; ++j) if (i != j) s += y_full[j];
36             Ay_local[li] = s;
37         }
38         double local_num = 0, local_den = 0;
39         for (int li = 0; li < local_rows; ++li) {
40             local_num += y_local[li] * Ay_local[li];
41             local_den += Ay_local[li] * Ay_local[li];
42         }
43         double num = 0, den = 0;
44         MPI_Allreduce(&local_num, &num, 1, MPI_DOUBLE, MPI_SUM,
45                       MPI_COMM_WORLD);
46         MPI_Allreduce(&local_den, &den, 1, MPI_DOUBLE, MPI_SUM,
47                       MPI_COMM_WORLD);
48         double tau = num / den;
49         for (int li = 0; li < local_rows; ++li)
50             x_local[li] -= tau * y_local[li];
51         iter++;
52     }
53     double t_end = MPI_Wtime();
54     if (rank == 0) {
55         std::cout << "Time: " << std::fixed << t_end - t_start;
56     }
57     MPI_Finalize();
58     return 0;
59 }

```

3 Характеристики устройства

Модель оборудования: MEGABOOK T14TA (TECNO T14TA)

Процессор: 11th Gen Intel(R) Core(TM) i5-115

Память: 16 ГиБ

Емкость диска: 512 ГБ

4 Время работы программ

1 вариант

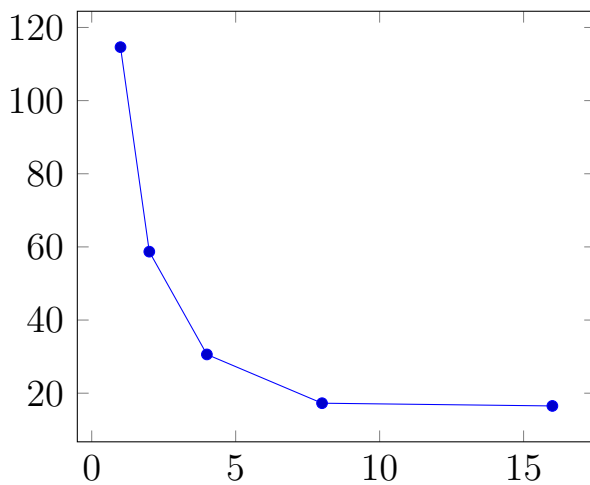
Время вычисления при использовании 1 ядра: 114.62 s

Время вычисления при использовании 2 ядер: 58.6987 s

Время вычисления при использовании 4 ядер: 30.5979 s

Время вычисления при использовании 8 ядер: 17.264 s

Время вычисления при использовании 16 ядер: 16.516 s



2 вариант

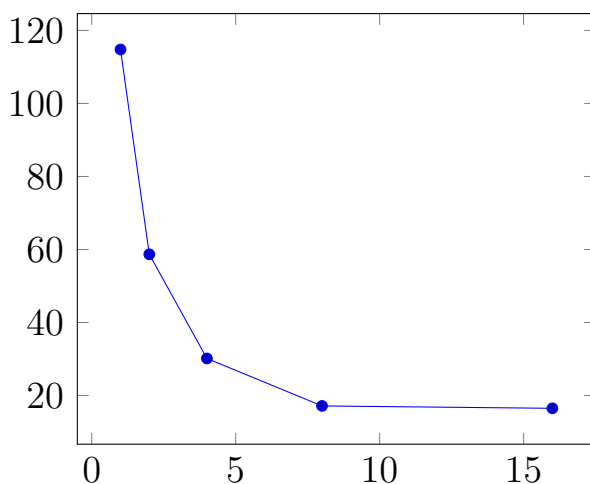
Время вычисления при использовании 1 ядра: 114.780174 s

Время вычисления при использовании 2 ядер: 58.684236 s

Время вычисления при использовании 4 ядер: 30.156448 s

Время вычисления при использовании 8 ядер: 17.151490 s

Время вычисления при использовании 16 ядер: 16.497 s



5 Вывод

В ходе выполнения лабораторной работы были разработаны две программы для решения системы линейных алгебраических уравнений методом минимальных невязок. В результате было получено, что вариант с распределенными векторами показывает несколько лучшую производительность по сравнению с вариантом с дублированными векторами. Также оба варианта программы демонстрируют существенное ускорение при увеличении числа ядер.